

```

# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

/kaggle/input/datasets/kyanyoga/sample-sales-data/
sales_data_sample.csv

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

# Style
sns.set(style='whitegrid')

```

1. LOAD DATASET

```

df =
pd.read_csv('/kaggle/input/datasets/kyanyoga/sample-sales-data/sales_d
ata_sample.csv', encoding='latin1')

# Display basic info
df.head()

```

ORDERNUMBER		QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	
SALES \					
0	10107	30	95.70	2	2871.00
1	10121	34	81.35	5	2765.90
2	10134	41	94.74	2	3884.34
3	10145	45	83.26	6	3746.70
4	10159	49	100.00	14	5205.27

ORDERDATE		STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	\
0	2/24/2003 0:00	Shipped	1	2	2003	...	
1	5/7/2003 0:00	Shipped	2	5	2003	...	
2	7/1/2003 0:00	Shipped	3	7	2003	...	
3	8/25/2003 0:00	Shipped	3	8	2003	...	
4	10/10/2003 0:00	Shipped	4	10	2003	...	

ADDRESSLINE1		ADDRESSLINE2	CITY	STATE	\
0	897 Long Airport Avenue	NaN	NYC	NY	
1	59 rue de l'Abbaye	NaN	Reims	NaN	
2	27 rue du Colonel Pierre Avia	NaN	Paris	NaN	
3	78934 Hillside Dr.	NaN	Pasadena	CA	
4	7734 Strong St.	NaN	San Francisco	CA	

POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME
DEALSIZE				
0	10022	USA	NaN	Yu Kwai
Small				
1	51100	France	EMEA	Henriot Paul
Small				
2	75508	France	EMEA	Da Cunha Daniel
Medium				
3	90003	USA	NaN	Young Julie
Medium				
4	NaN	USA	NaN	Brown Julie
Medium				

[5 rows x 25 columns]

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#    Column                Non-Null Count  Dtype
---  -
0    ORDERNUMBER            2823 non-null   int64
1    QUANTITYORDERED        2823 non-null   int64
```

2	PRICEEACH	2823	non-null	float64
3	ORDERLINENUMBER	2823	non-null	int64
4	SALES	2823	non-null	float64
5	ORDERDATE	2823	non-null	object
6	STATUS	2823	non-null	object
7	QTR_ID	2823	non-null	int64
8	MONTH_ID	2823	non-null	int64
9	YEAR_ID	2823	non-null	int64
10	PRODUCTLINE	2823	non-null	object
11	MSRP	2823	non-null	int64
12	PRODUCTCODE	2823	non-null	object
13	CUSTOMERNAME	2823	non-null	object
14	PHONE	2823	non-null	object
15	ADDRESSLINE1	2823	non-null	object
16	ADDRESSLINE2	302	non-null	object
17	CITY	2823	non-null	object
18	STATE	1337	non-null	object
19	POSTALCODE	2747	non-null	object
20	COUNTRY	2823	non-null	object
21	TERRITORY	1749	non-null	object
22	CONTACTLASTNAME	2823	non-null	object
23	CONTACTFIRSTNAME	2823	non-null	object
24	DEALSIZE	2823	non-null	object

dtypes: float64(2), int64(7), object(16)

memory usage: 551.5+ KB

df.describe()

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	\
count	2823.000000	2823.000000	2823.000000	2823.000000	
mean	10258.725115	35.092809	83.658544	6.466171	
std	92.085478	9.741443	20.174277	4.225841	
min	10100.000000	6.000000	26.880000	1.000000	
25%	10180.000000	27.000000	68.860000	3.000000	
50%	10262.000000	35.000000	95.700000	6.000000	
75%	10333.500000	43.000000	100.000000	9.000000	
max	10425.000000	97.000000	100.000000	18.000000	

	SALES	QTR_ID	MONTH_ID	YEAR_ID	MSRP
count	2823.000000	2823.000000	2823.000000	2823.000000	2823.000000
mean	3553.889072	2.717676	7.092455	2003.81509	100.715551
std	1841.865106	1.203878	3.656633	0.69967	40.187912
min	482.130000	1.000000	1.000000	2003.00000	33.000000
25%	2203.430000	2.000000	4.000000	2003.00000	68.000000

50%	3184.800000	3.000000	8.000000	2004.00000	99.000000
75%	4508.000000	4.000000	11.000000	2004.00000	124.000000
max	14082.800000	4.000000	12.000000	2005.00000	214.000000

2. DATA CLEANING

```
# Drop irrelevant columns
drop_cols = [
    'ADDRESSLINE1', 'ADDRESSLINE2', 'CONTACTFIRSTNAME',
    'CONTACTLASTNAME', 'POSTALCODE', 'ORDERNUMBER'
]
df.drop(columns=drop_cols, inplace=True)

# Handle missing values
df['STATE'] = df['STATE'].fillna('Unknown')
df['TERRITORY'] = df['TERRITORY'].fillna('Unknown')

# Convert date
df['ORDERDATE'] = pd.to_datetime(df['ORDERDATE'])
```

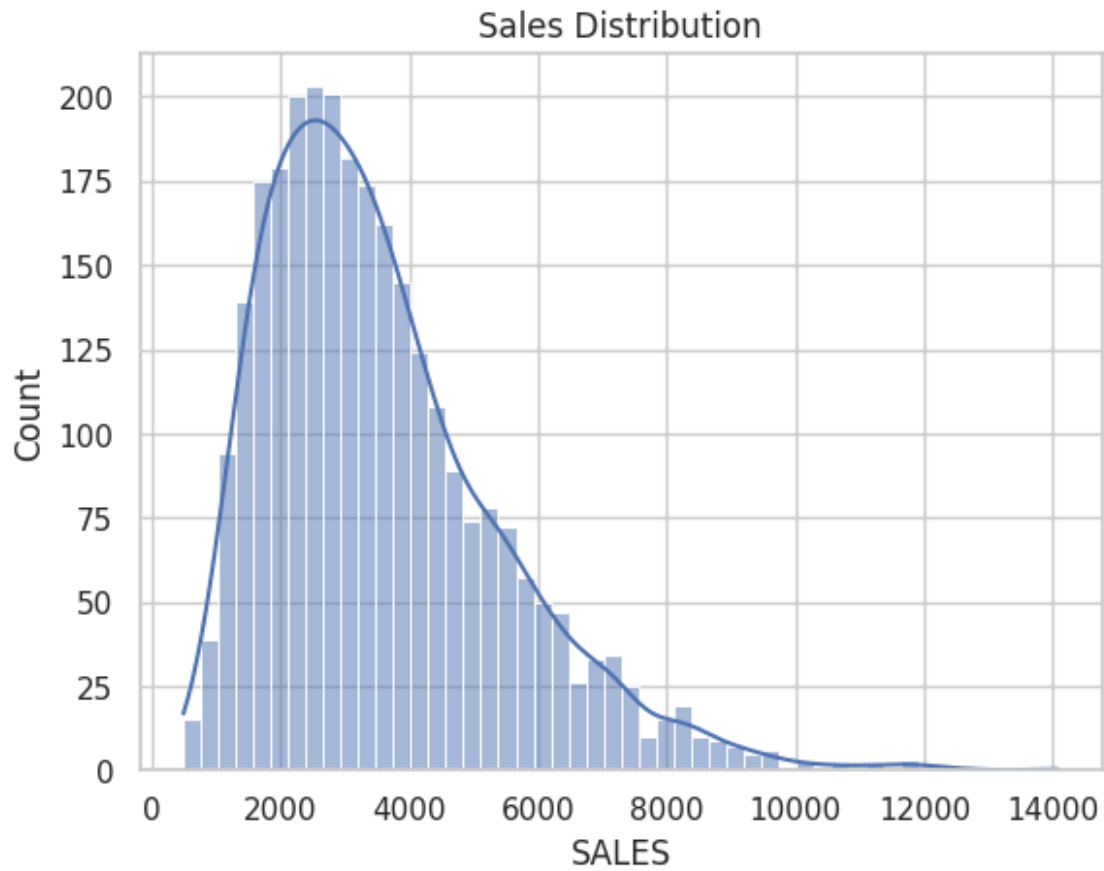
3. FEATURE ENGINEERING

```
df['YEAR'] = df['ORDERDATE'].dt.year
df['MONTH'] = df['ORDERDATE'].dt.month

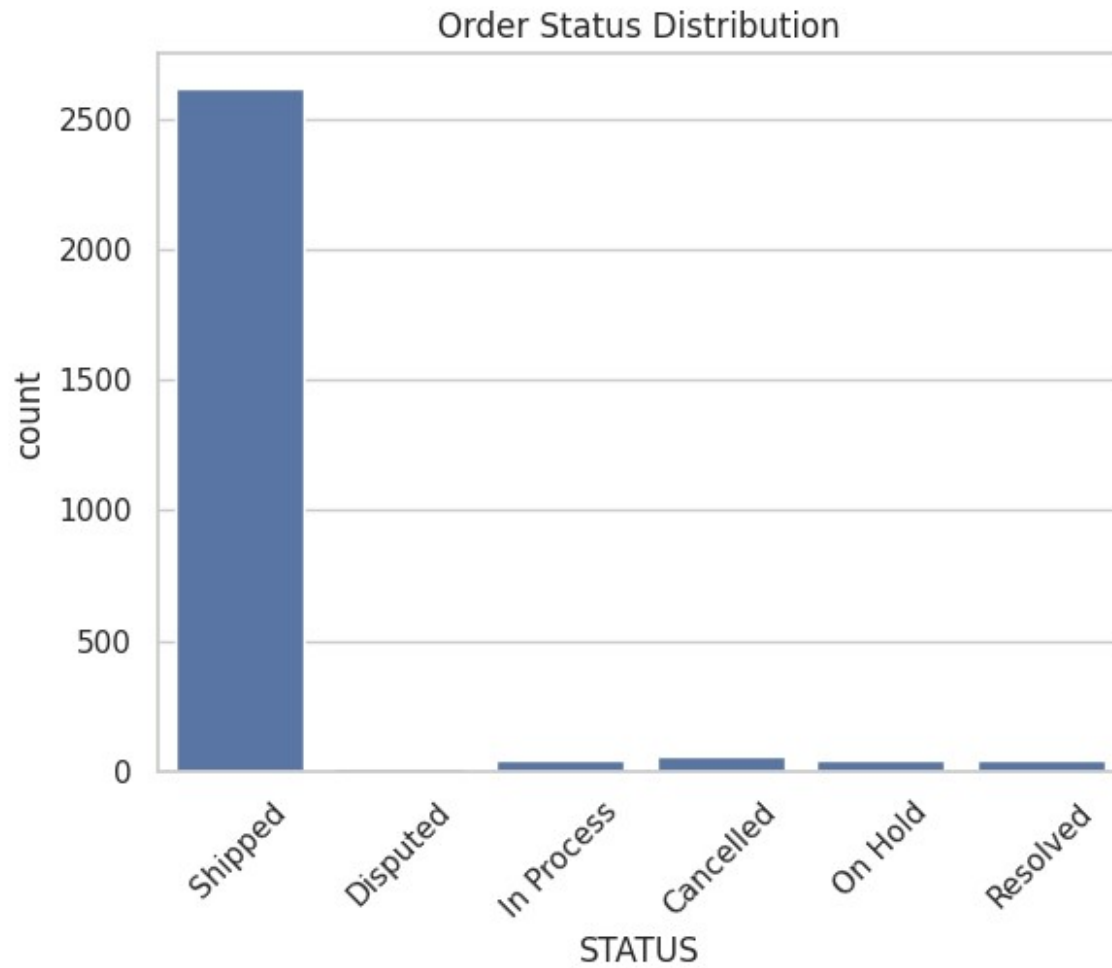
df.drop('ORDERDATE', axis=1, inplace=True)
```

4. EXPLORATORY DATA ANALYSIS (EDA)

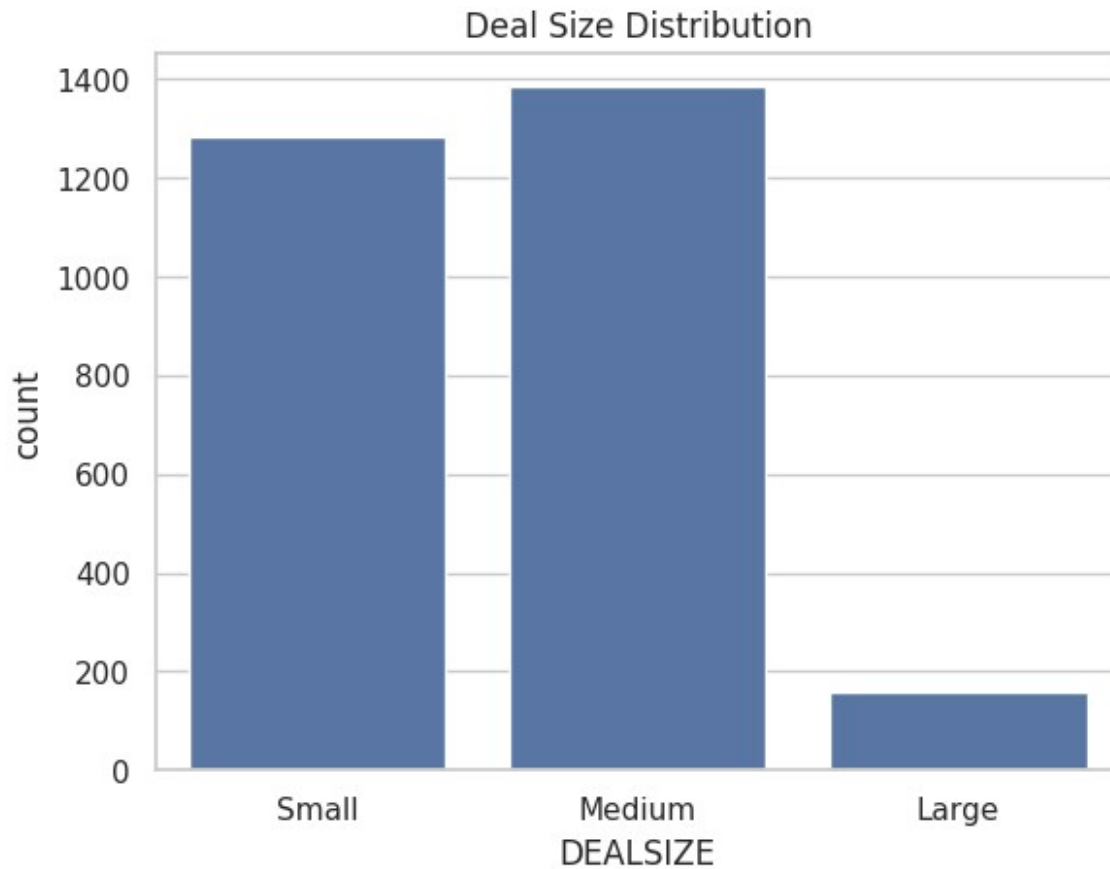
```
# Sales distribution
plt.figure()
sns.histplot(df['SALES'], bins=50, kde=True)
plt.title("Sales Distribution")
plt.show()
```



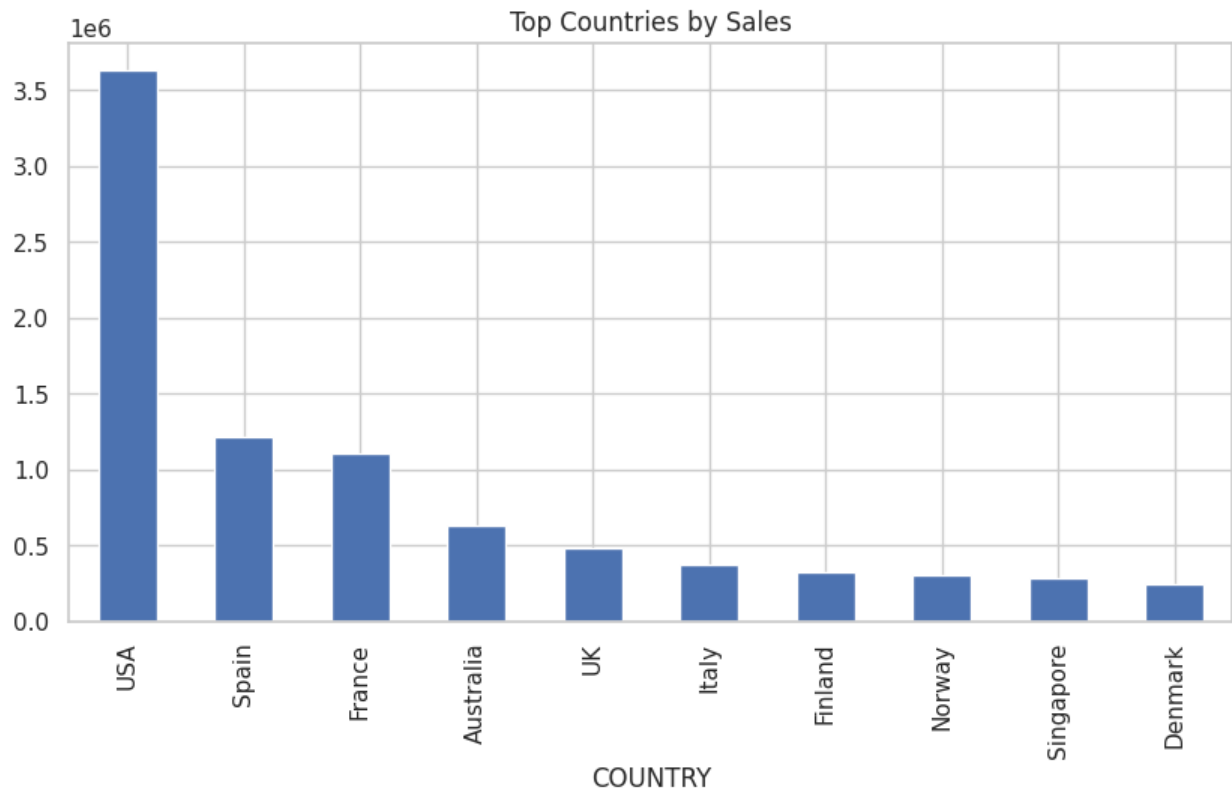
```
# Status distribution  
plt.figure()  
sns.countplot(x='STATUS', data=df)  
plt.title("Order Status Distribution")  
plt.xticks(rotation=45)  
plt.show()
```



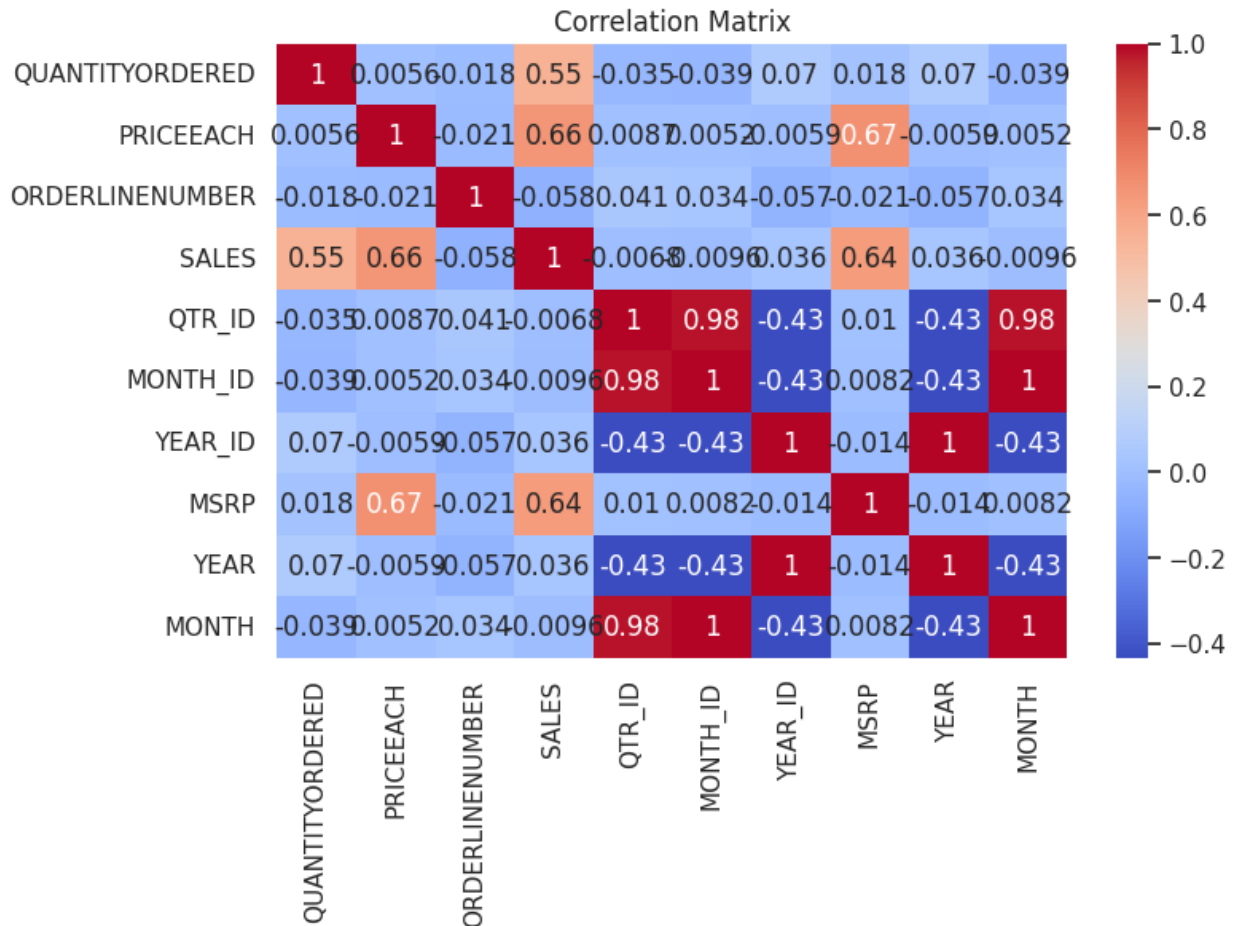
```
# Deal Size count  
plt.figure()  
sns.countplot(x='DEALSIZE', data=df)  
plt.title("Deal Size Distribution")  
plt.show()
```



```
# Sales by Country
plt.figure(figsize=(10,5))
df.groupby('COUNTRY')
['SALES'].sum().sort_values(ascending=False).head(10).plot(kind='bar')
plt.title("Top Countries by Sales")
plt.show()
```



```
# Correlation heatmap
plt.figure(figsize=(8,5))
numeric_df = df.select_dtypes(include=np.number)
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

5. ENCODING CATEGORICAL DATA

```
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

X = df.drop('STATUS', axis=1)
y = df['STATUS']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

6. RANDOM FOREST MODEL

```
rf = RandomForestClassifier(n_estimators=150, random_state=42)
rf.fit(X_train, y_train)

RandomForestClassifier(n_estimators=150, random_state=42)

y_pred = rf.predict(X_test)
```

7. EVALUATION

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

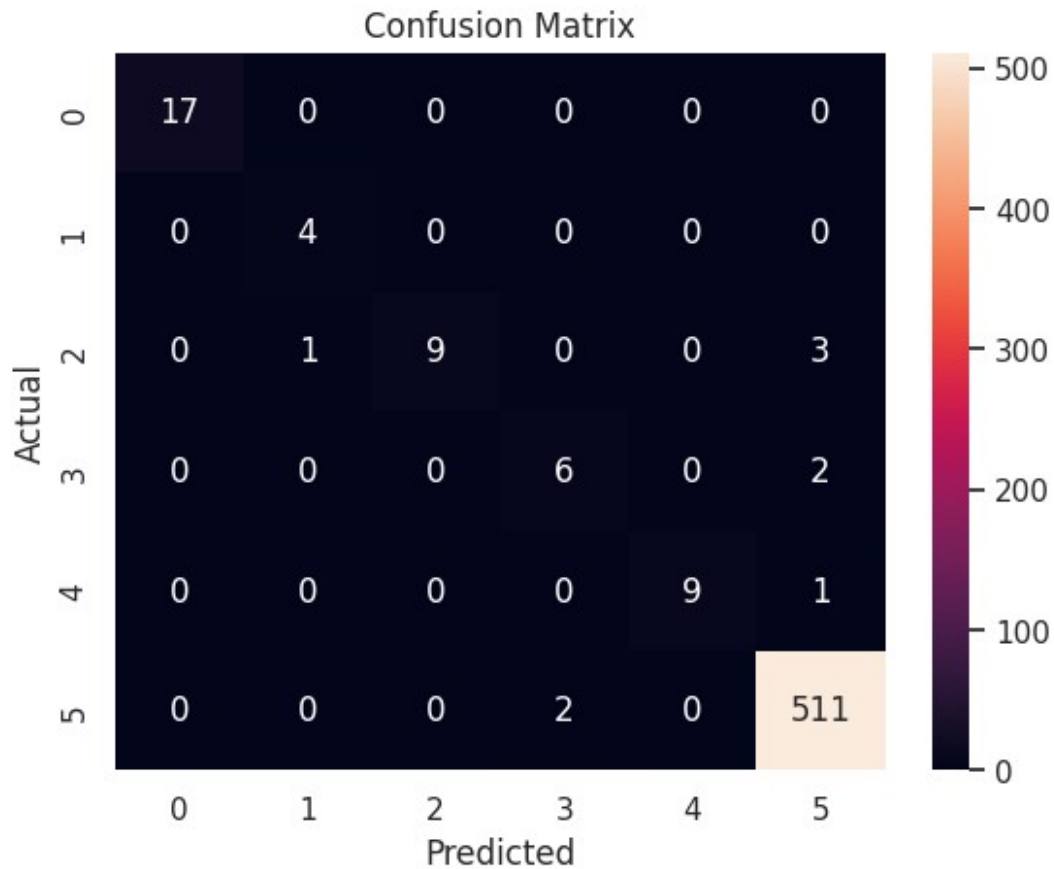
Accuracy: 0.984070796460177

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	17
1	0.80	1.00	0.89	4
2	1.00	0.69	0.82	13
3	0.75	0.75	0.75	8
4	1.00	0.90	0.95	10
5	0.99	1.00	0.99	513
accuracy			0.98	565
macro avg	0.92	0.89	0.90	565
weighted avg	0.98	0.98	0.98	565

```
# Confusion Matrix
```

```
plt.figure()
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

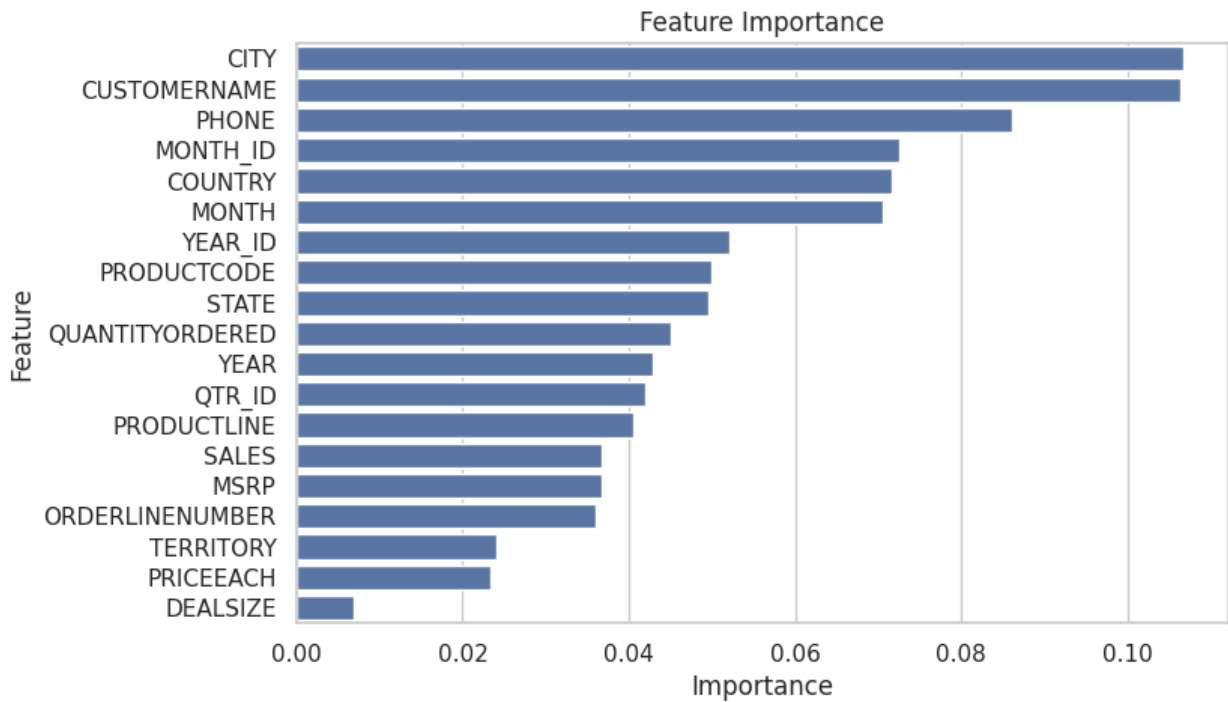


8. FEATURE IMPORTANCE

```
feat_imp = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf.feature_importances_
}).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(8,5))
sns.barplot(x='Importance', y='Feature', data=feat_imp)
plt.title("Feature Importance")
plt.show()

print(feat_imp.head(10))
```



	Feature	Importance
12	CITY	0.106821
10	CUSTOMERNAME	0.106324
11	PHONE	0.086068
5	MONTH_ID	0.072556
14	COUNTRY	0.071628
18	MONTH	0.070572
6	YEAR_ID	0.052071
9	PRODUCTCODE	0.050013
13	STATE	0.049579
0	QUANTITYORDERED	0.045024

CONCLUSION

The Random Forest model achieved very high accuracy (98.4%), indicating strong performance in classifying order status. The model effectively handled both numerical and categorical features, with key factors such as CITY, CUSTOMERNAME, PHONE, and time-related features contributing most to predictions.

The classification report shows excellent performance for major classes, though minor classes have slightly lower recall due to imbalance. Overall, the model is robust, reliable, and suitable for business intelligence applications, helping organizations predict order outcomes and improve decision-making.